APIs y Arquitectura Web

Emmanuel Cetina Gomez

1. Arquitectura del Software y Modelo Multicapa

¿Qué es la Arquitectura de Software?

Conjunto de elementos de diseño e implementación de estructuras de alto nivel para satisfacer requisitos funcionales y no funcionales.

¿Qué es el Modelo Multicapa?

Organización en niveles de abstracción (capas). Cada capa se apoya en la inferior y presta servicios a la superior.

Arquitectura Abierta vs. Cerrada

Abierta: cualquier capa accede a cualquier otra. Cerrada (recomendada): solo se interactúa con la capa inmediatamente inferior.

3 Capas Principales

1. Presentación – Interfaz con el usuario.
2. Lógica de Negocio – Funcionalidad del sistema.
3. Acceso a Datos – Persistencia en BD, XML, archivos.

Ventajas

Componentes separados · Trabajo en paralelo · Reutilización de código · Distribución de carga.

Desventajas

Mayor cantidad de código · Tiempos de ejecución ligeramente superiores.

2. Arquitectura Web: Cliente-Servidor y la Separación Frontend/Backend

CLIENTE - SERVIDOR

Cliente: solicita la información (tu navegador).
Servidor: valida y entrega los recursos.
Hoy: AJAX y WebSockets permiten actualizar solo partes de la página.

EVOLUCIÓN HISTÓRICA

90s-2008: Monolítico (PHP hacía todo).
2008-2010: HTML5 potencia el navegador. Nace la separación Frontend/Backend.
Hoy: WebAssembly, frameworks modernos y microservicios.

FRONTEND (Navegador)

Maneja la lógica del cliente y la interfaz. Frameworks: Angular, React, Vue. Mito: no necesita saber diseñar.

BACKEND (Servidor)

Lógica de negocio, base de datos, seguridad, rendimiento. Crea la API para que el frontend consuma. Tendencia: microservicios y BaaS (Firebase).

LA API: El puente de conexión

Permite que Frontend (React, Angular) se comunique con cualquier Backend sin importar el lenguaje.

3. ¿Qué es una API?

API = Application Programming Interface

Conjunto de definiciones y protocolos que permiten que dos aplicaciones se comuniquen entre sí, sin exponer el código interno.

Privadas / Públicas

Pueden ser internas (empresa), solo para partners o públicas para cualquier desarrollador.

Para qué sirven

Ahorrar tiempo y dinero reutilizando servicios ya creados: pagos, mapas, login social, almacenamiento en nube...

Ejemplos cotidianos

Login con Facebook · Pagar con tarjeta online · Ver un mapa en una app · Publicar en redes sociales.

La API es un "túnel de comunicación" entre lo que el usuario quiere hacer y lo que la plataforma ejecuta internamente, devolviendo solo el resultado necesario.

4. Web Services y el Estándar REST

Web Service vs API

Un Web Service es un tipo de API que opera mediante HTTP. Todo Web Service es API, pero no toda API es Web Service (ej. API del kernel de Linux).

¿Qué es REST?

Representational State Transfer. Conjunto de normas para usar las peticiones HTTP de forma estandarizada y uniforme entre sistemas.

Beneficios de una API REST

Simple y Estandarizada

Enfoque uniforme: no hay que decidir formato o estructura cada vez.

Escalable y Sin Estado

Fácil de modificar conforme crece. No rastrea el estado de los datos entre peticiones.

Alto Rendimiento

Soporta caché, mantiene velocidad incluso en sistemas complejos.

Endpoint ejemplo: icecream.com/api/flavors → /api/ indica la porción de API · flavors es el recurso.

Comunicación: REQUEST (cliente→servidor) con método HTTP + endpoint + body + headers | RESPONSE (servidor→cliente) en JSON.

5. Métodos HTTP y Operaciones CRUD

GET **READ**	**Obtener datos** *GET /api/flavors → Retorna lista de sabores en JSON*
POST **CREATE**	**Crear nuevo recurso** *POST /api/flavors + body: "Restful Raspberry" → Crea ID nuevo*
PUT **UPDATE**	**Reemplazar recurso existente** *PUT /api/flavors/1 + body: "Chocolate" → Reemplaza el ID 1*
DELETE **DELETE**	**Eliminar recurso** *DELETE /api/flavors/1 → Elimina el sabor con ID 1*

6. JSON – El Lenguaje de las APIs

JavaScript Object Notation

Formato ligero y legible para intercambiar datos. Alternativa a XML, más limpio y con soporte nativo en JS.

Reglas clave de sintaxis

- Claves siempre entre "comillas dobles"
- Separar pares con : y elementos con ,
- Sin coma al final del último elemento
- Raíz: un objeto { } o un arreglo [ ]

Tipos de datos soportados

String
"nombre": "Pikachu"

Boolean
"activo": true

Array
"tipos": ["eléctrico"]

Number
"nivel": 25

Null
"ceo": null

Object
{ "id": 1, "nombre": "..." }

JSON.parse(string) en JavaScript convierte el texto recibido de una API en un objeto manipulable. Compatible con casi todos los lenguajes de programación.

7. Patrón de Diseño MVC (Modelo-Vista-Controlador)

CONTROLADOR	MODELO	VISTA
Intermediario. Recibe la petición del cliente, le dice al Modelo qué hacer y a la Vista qué mostrar. No contiene lógica compleja.	Maneja toda la lógica de datos. Interactúa directamente con la base de datos: guardar, actualizar, eliminar, consultar.	Solo se encarga de la presentación. Recibe datos del Controlador y los renderiza en HTML para el usuario.

Regla de Oro: El Modelo y la Vista NUNCA interactúan directamente. Toda comunicación pasa por el Controlador.

Flujo de una petición:

Usuario → Controlador → Modelo (BD) → Controlador → Vista (HTML) → Respuesta al usuario